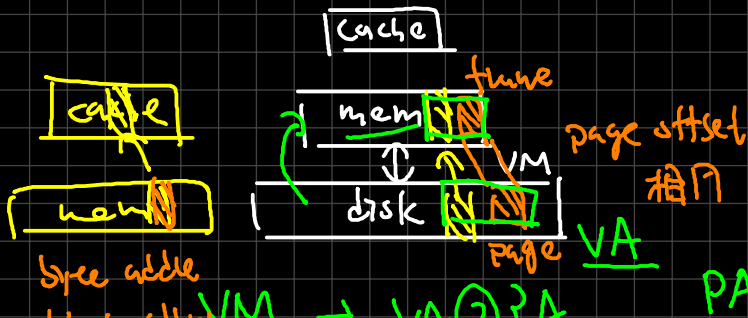


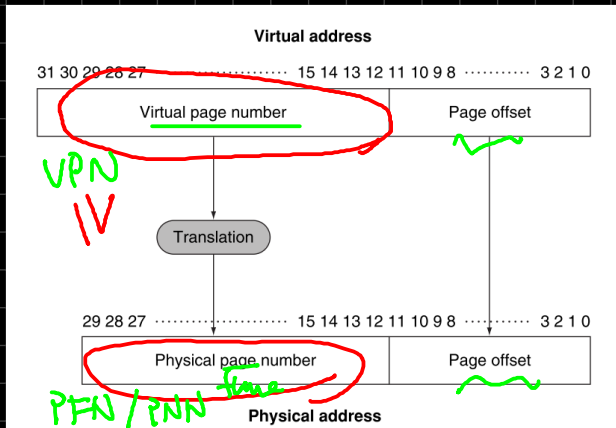
Virtual Memory : Reasons:

✓ (1) main memory 当作 disk 的 cache



* 大小 (ex. 1KB)
page 是 VM 的 basic unit
(1/2 的 cache 的 block)

(2) translation
VM → VA → PA
data



ex. page offset = 4 byte 16
Disk 虚拟 addr. = 0x1234 ABCD
new physical addr. = 0x321 ABCD

(3) 虚拟 page (disk) 与 物理 page (memory) 的

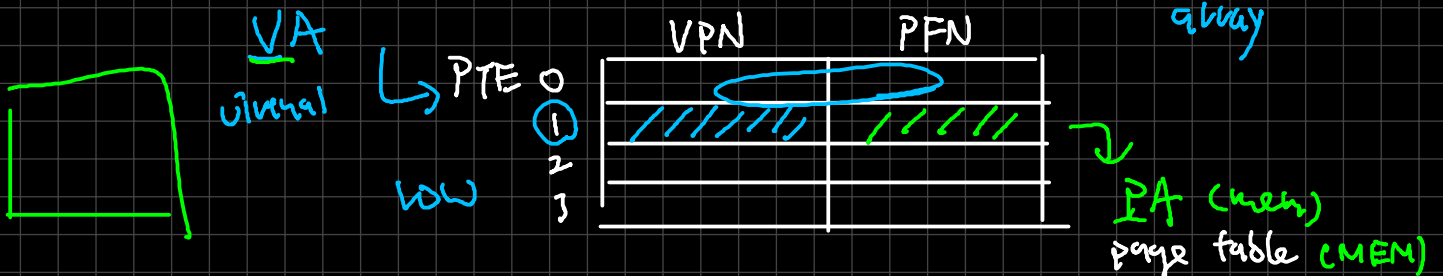
→ VM 是 fully-associative &

每个 Virtual Page 都对应一个 Physical frame

VPN (Virtual Page Number)	PFN (Physical Frame Number)
0x12	0xab

page table entry (PTE)

→ page table entry (PTE) 存储在 Page table 中



(4) paging

Step 1: 把 VA 拆成 PA

$(\text{VPN} \rightarrow \text{PFN}) + \text{page offset}$

ex. page offset = 4 byte

virtual addr. = 0x1234 ABCD

PA physical addr. = 0x321 ABCD

Step 2:

VA $\rightarrow$ mem

① hit = find address by translation VA ✓

② miss (page fault) = 去 disk 找 data 存 memory

VM OS interrupt $\rightarrow$ process context switch

(running state $\rightarrow$ waiting state)

$\rightarrow$ return from I/O complete

(waiting state $\rightarrow$ ready state)

$\rightarrow$ CPU scheduler

MMU

base page table eg

VPN	PFN

cache
 $\downarrow \uparrow$
mem
back through

mem replace
 $\downarrow \uparrow$
disk
write back (dirty-bit=1)

(5) VM 写回

write through penalty 大 (disk I/O 大)

dirty-bit

(6) LRU (Least Recently Used) = 最不被使用的页

page table 替换

(4) 5 5 5 8

(7)

	cache	VM
		0 1 future page page
basic unit	block	page (通常 4K ex. 1KB)
hit/miss	hit/miss	hit / <u>page fault</u> (miss)
associative	direct mapped / set associative / fully associative	<u>fully</u> associative ($\because$ 任何 64 virtual page 都可以 map 到 physical frame)

hit/miss eg

VA $\rightarrow$ PA x

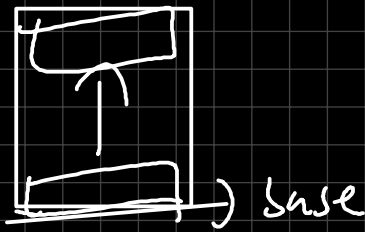
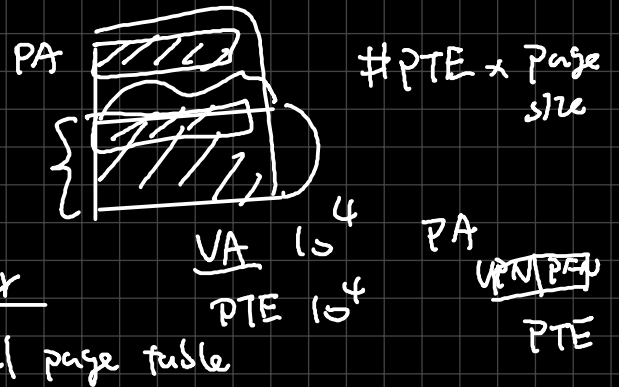
PTE

VPN/PFN

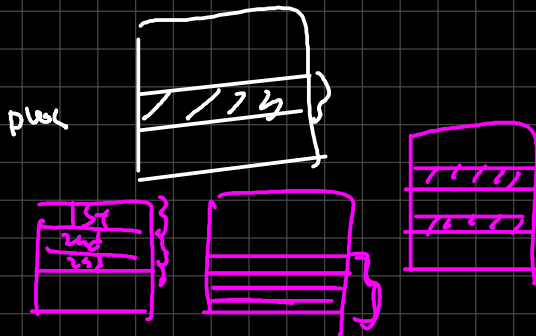
page
table
translation x

(8) problem

(a) page table too



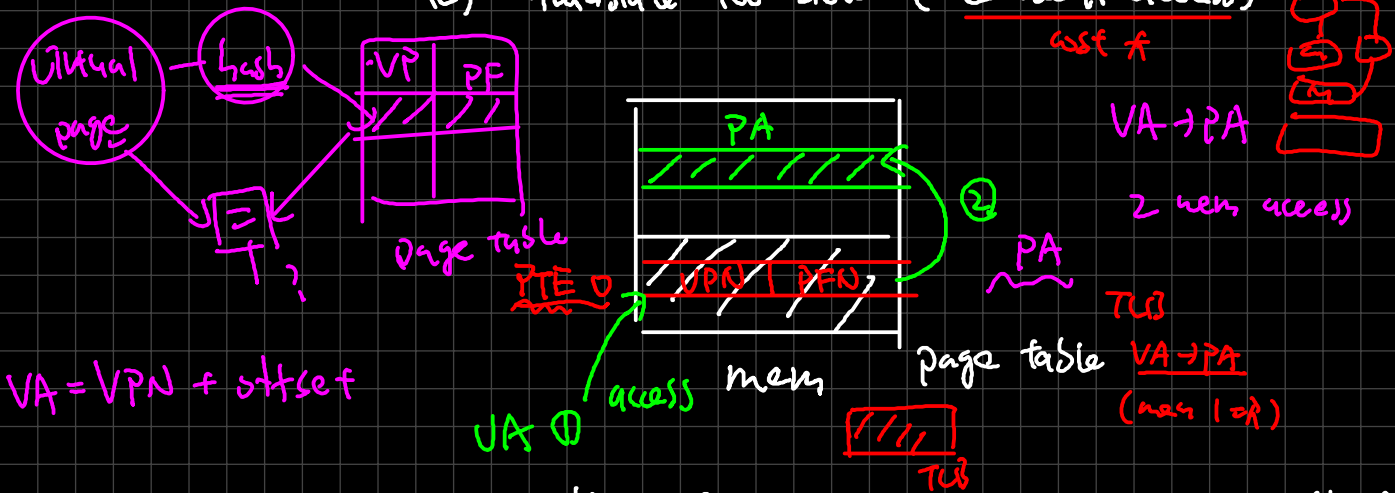
sol) base register
multi-level page table



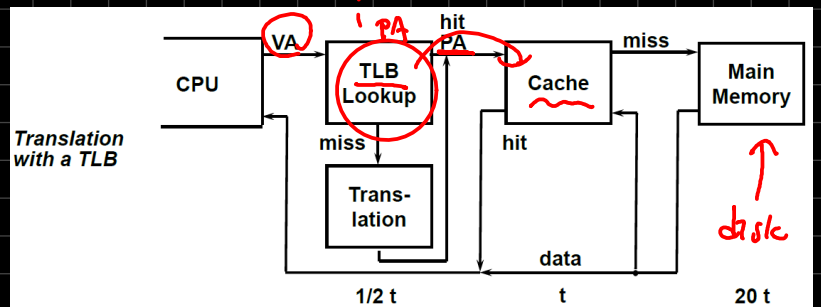
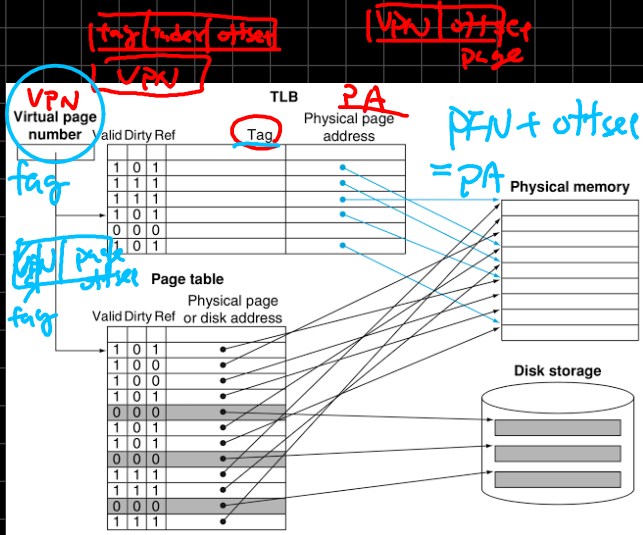
VA → PA

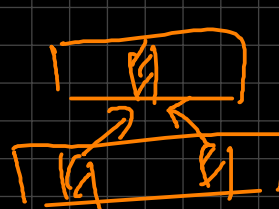
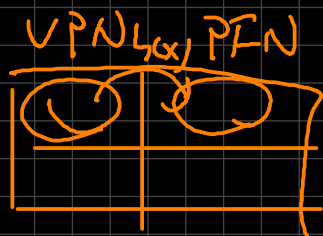
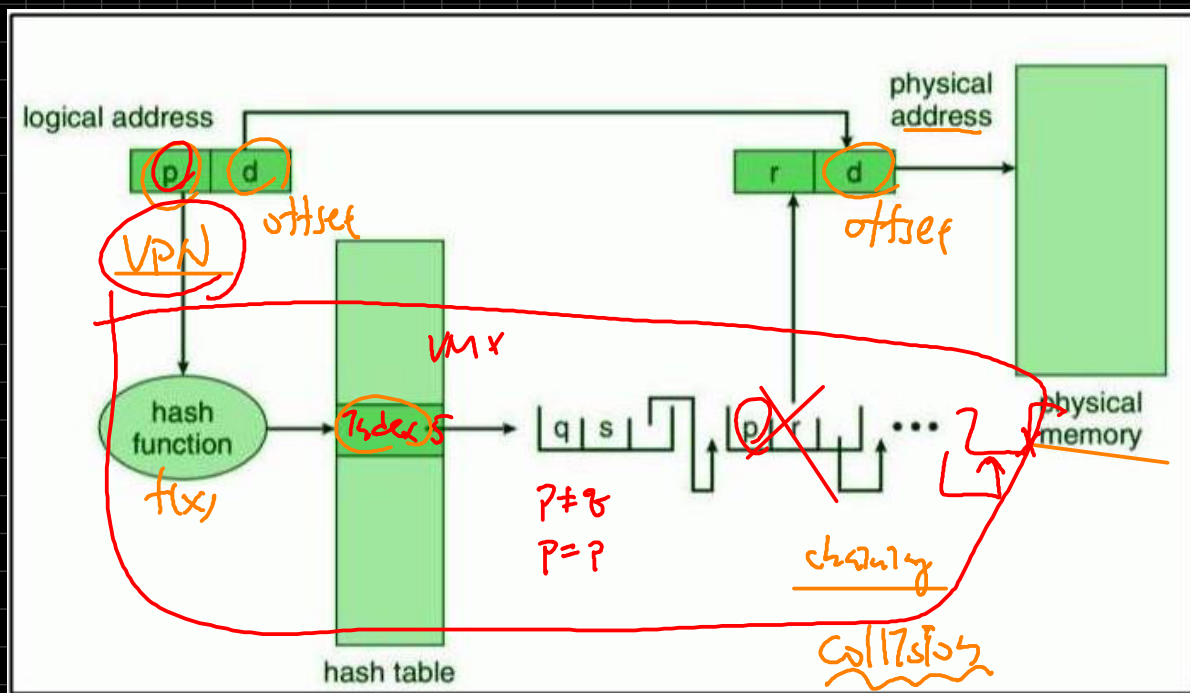
hashing: Inverted page table

(b) translate too slow (2 mem access)



sol) TLB (Translation Lookaside Buffer)





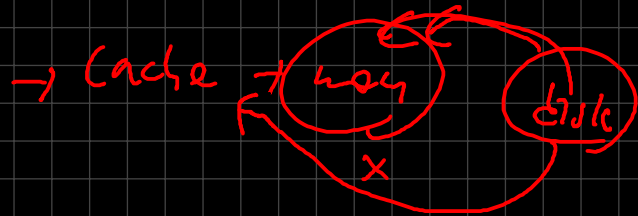
AM
open & close
chain
 (7) per putting

TLB L7E

(174)

- cache {
- ① read L7E → good!
 - ② write L7E → dirty-bit = 1 (write back to page table on replacement)
write back

TLB miss



Case 1 = page in MEM

→ load PTE from MEM, then dirty
(page table)

Case 2 = page NOT in MEM

→ 不在 TLB, 不在 MEM ⇒ 在 disk, then dirty
(OS interrupt handle)

22. A system has a 16-bit virtual address size with 256B pages and a 16-bit byte addressing physical memory with physically index 2-way set associative data cache (LRU replacement policy) of 512B cache size and 16B blocks. The first six accesses to the page table are: 0x1332, 0xad58, 0x0162, 0x2ea9, 0x813d, 0x9f5a. The system uses a single level page table. Assume the TLB and data cache are initially empty and their contents have been updated accordingly by the first six accesses (page table is shown below; VPN: virtual page number, PPN: physical page number). The CPU includes a fully associative TLB with 2 entries and an LRU replacement policy. Data reads from the following virtual addresses are performed (in the order listed): 0x813f, 0x136f, 0x2e5f, 0x9f50, 0x1370. Which ones of following statements are correct?

VPN	PPN	VPN	PPN
0x01	0x27	0x81	0x8a
0x13	0x45	0x9f	0xcd
0x2e	0xe8	0xad	0x7e

page table

- (a) The required numbers of bits for virtual page number and page offset in virtual address are all 8 bits.
- (b) The required number of bits for index and tag in physical address are 4 and 8 bits respectively.
- (c) Data access to 0x813f has access hit in TLB and cache.
- (d) Data access to 0x9f50 has access miss in TLB and access hit in cache.
- (e) Data access to 0x1370 has access miss in TLB and cache.

virtual:

addr = 16 bit

page = 256 B

physical:

16 bit byte addressing

2-way set associative (with LRU)

512 B cache size

16 B blocks

single level page table



6 access to page table: 0x1332, 0xad58, 0x0162, 0x2ea9, 0x813d, 0x9f5a

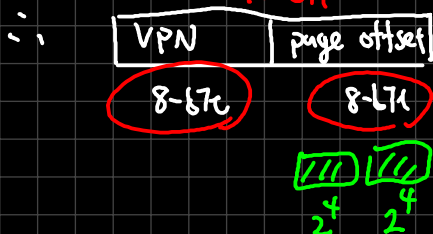
CPU:

fully associative TLB, with 2 entry, LRU

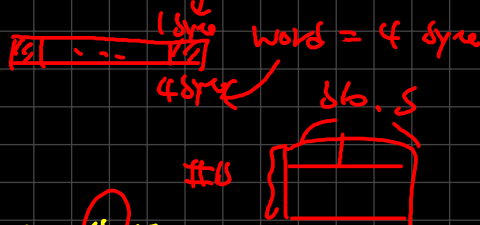
data read:

0x813f, 0x136f, 0x2e5f, 0x9f50, 0x1370

(a) page size = 256 B = 2^8 B



(b) cache size = 512 B, 16 Bt byte addressing
 block size = 16 B
 2-way associative

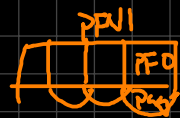


blocks = $\frac{512}{16} = 32$

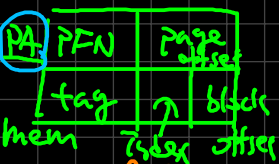
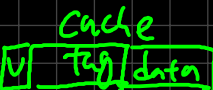
set = $\frac{32}{2} = 16 = 2^4$, index = 4 Bt

block size = 16 = 2^4 B, offset = 4 Bt

tag = 16 - 4 - 4 = 8 Bt



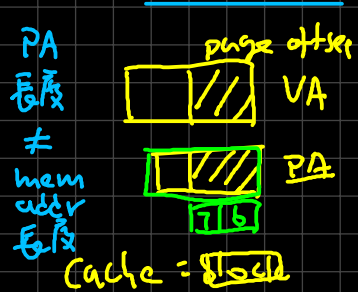
6 access to page table: 0x1332, 0xad58, 0x01b2, 0x2ea9, 0x813d, 0x9f5a



VPN	PPN	VPN	PPN
0x01	0x27	0x81	0x8a
0x13	0x45	0x9f	0xcd
0x2e	0xe8	0xad	0x7e



block offset
 page offset
 VIPT



VA	PA	cache set / tag
0x1332	0x4532	set 3, tag 45
0xad58	0x7e58	set 5, tag 7e
0x01b2	0x27b2	set 6, tag 27
0x2ea9	0xe8a9	set 10, tag e8
0x813d	0x8a3d	set 3, tag 8a
0x9f5a	0xcd5a	set 5, tag cd

TLB 只有 2 entry, 6 access 之後 只剩 81, 9f

Translation Lookaside Buffer
 Cache 16 Bt

Set	2way
3	tag 45, tag 8a
5	tag 7e, tag cd
6	tag 27
10	tag e8

x 13, ad
 01, ad x
 x 01, 2e
 81, 2e x
 x 81, 9f
 VPN

再找 data lead:

0x813f, 0x136f, 0x2e5f, 0x9f50, 0x1370

✓ ① $0x813f$
VPN

TLB 有 $81 \rightarrow$ hit

换成 PA $\rightarrow 0x8a3f$

cache: tag = $8a$
 set = 3

$\therefore$ hit

(C) (TLB, cache) = (hit, hit)

TLB
 $81, 9f$

Cache 128×4

Set

3	tag 45, tag 8a
5	tag 7e, tag cd
6	tag 2f
10	tag e8

② $0x136f$
VPN

TLB 没有 13 $\rightarrow$ miss ($13 \rightarrow 45$)

换成 PA $\rightarrow 0x456f$

cache: tag = 45
 set = 6

$\therefore$ miss

LRU

TLB 更新: $81, 13$; cache 更新: set 6 的 tag 45

TLB
 $81, 9f$

VPN

PPN

Cache 128×4

Set

3	tag 45, tag 8a
5	tag 7e, tag cd
6	tag 2f, tag 45
10	tag e8

③ $0x2e5f$
VPN

TLB 没有 2e $\rightarrow$ miss ($2e \rightarrow e8$)

换成 PA $\rightarrow 0xe85f$

cache: tag = e8
 set = 5

$\therefore$ miss

LRU

TLB 更新: $2e, 13$; cache 更新: set 5 的 tag 7e 换成 tag e8

TLB
 $81, 13$

VPN

PPN

Cache 128×4

Set

3	tag 45, tag 8a
5	tag 7e, tag cd tag e8
6	tag 2f, tag 45
10	tag e8

④ $0x9f50$
VPN

TLB 没有 9f $\rightarrow$ miss ($9f \rightarrow cd$)

换成 PA $\rightarrow 0xcd50$

cache: tag = cd
 set = 5

$\therefore$ hit

LRU

TLB 更新: $2e, 9f$ (d) (TLB, cache) = (miss, hit)

TLB
 $2e, 13$

VPN

PPN

Cache 128×4

Set

3	tag 45, tag 8a
5	tag e8, tag cd
6	tag 2f
10	tag e8

Q 0x1370
VPN

TLB
2e, 9f

#16

TLB 没有 13 → miss (13 → 45)

换成 PA → 0x4570

cache: tag = 45

Set = 7

miss

TLB 更新 = 13, 9f ; cache 更新 = 4570 Set 13, tag 9f

(e) (TLB, cache) = (miss, miss)

next?

台大 100

Cache 128 组

Set

3 tag 45, tag 8a
5 tag 9e, tag cd
6 tag 2f
10 tag e8
7 tag 9f, 45

6. (20 pts) You are given a bus-based multiprocessor system which implements a snooping-based cache coherency protocol. There are two processors P1, P2 in the system. Each processor has its own L1 cache. The memory system specification is given below:

- L1 Cache: physical addressed, 64KB, direct-mapped, 32B block size, write-back cache
- Virtual Memory: 8K page, 1 GB virtual address space
- TLB: fully associative, 128 entries
- 64 MB of addressable physical memory.

x1, x2, and x3 represent one-word variables. The physical addresses (represented in hex) of x1, x2 and x3 are 0x00AF0804, 0x00AF0808, and 0x00AE0804, respectively. Answer the following questions considering the possible events: TLB-hit/miss, page-table hit/miss, L1-hit/miss, write-back, invalidate.

a. (3 pts) What are the sizes of the tag and data arrays of the TLB?

b. (2 pts) Please write down the correct order of the events listed above for detecting a page fault. Note that some events may not occur.

c. (15 pts) Assume TLB hits for all the memory requests. X1 and x2 are in the L1 caches of P1 and P2. Show the bus activities for each memory request.

Time	P1	P2	Bus activity
1	Write x1		
2		Read x2	
3	Write x1		
4		Read x3	
5	Read x2		

Snooping
I/O CA

